Tweet Sentiment Label Quality: A Text Classification Analysis

David Ewing · Student ID: 82171165

1 Problem Statement

This project examines how well an automated text classifier can reproduce the sentiment labels applied to a large sample of tweets. The dataset—Cardiff NLP's tweet_eval sentiment subset—has 45,615 tweets, each tagged as negative, neutral, or positive. Labelling was conducted as part of the SemEval 2017 Task 4A shared task on sentiment analysis in Twitter (Rosenthal et al., 2017), and the labels are widely used as an NLP benchmark.

My central question is: how reliably can a text classifier reproduce these labels, and what does classifier performance reveal about the quality and consistency of the annotations? This matters because weak classification performance may reflect not just the difficulty of the task but genuine inconsistency in how the labels were assigned. A classifier that performs poorly on neutral tweets may be revealing that the boundary between neutral and mildly positive or mildly negative is genuinely ambiguous — that different annotators drew that line differently. Put differently, low F1 may be evidence of noisy labels as much as evidence of an inadequate model. Both can be true at once.

Four smaller questions guide this analysis:

1. What kinds of problems are present in the labelling of tweets?
2. What types of tweets are systematically hard to classify?
3. What features are most helpful for distinguishing sentiment classes?
4. Does removing the neutral class improve classifier performance? What does this say about how consistent the neutral label is?

Annotated datasets are expensive to produce. My sense is that once a benchmark is established, it tends to be used repeatedly—models are compared using the same labels, which are rarely questioned. If I follow that reasoning, gold-standard labels that contain systematic noise mean every model comparison on that benchmark inherits that noise. A model that achieves a higher F1 than a competitor may simply be better at reproducing the particular pattern of annotator disagreement baked into the dataset, not better at genuine sentiment classification. That is why I think being explicit about label quality matters—it is a methodological obligation, not just an academic nicety.

I went with macro average F1—seems like the only fair way to score this mess. The validation split has 2,000 tweets; I use that as my test set from start to finish. The training data is lopsided—neutral tweets make up about 45%. To fix that, I chopped everything down with random undersampling. Every class ends up with 7,093 tweets—so 21,279 in total. I didn't want to oversample; that just repeats or makes up new examples, which felt like a bad idea with noisy labels floating around. Macro F1 treats all three classes the same. That matters to me, since I'm after differences between classes, not just overall hits. I also report per-class precision and recall. Accuracy? I skipped it. It's a trap in imbalanced setups—just rewards guessing the big class.

2 Dataset

Let me be blunt—this next bit? It's going to sound more like me talking than like a committee-approved report. Some things just feel off, and the dataset has quirks I can't ignore.

So, here's what I'm working with: the sentiment subset of the Cardiff NLP tweet_eval benchmark (Barbieri et al., 2020). Straight from Twitter, with all the chaos you'd expect. Annotators labeled tweets for SemEval 2017 Task 4A—teams racing to pin down sentiment in a few words. Labels are negative, neutral, and positive, all handed out by a crowd on the internet. Majority vote wins. Sometimes I look at that and think, is that really enough? I doubt it.

Crowdsourcing means you get Mechanical Turk workers reading tweets and slapping on a label (Rosenthal et al., 2017). Each tweet goes through at least three people. Whatever the majority says, that's the label. Fast, cheap, but messy. Annotators butt heads. Sometimes the majority label is basically a coin toss. When I see that, I start doubting the so-called "ground truth":

- a mildly sarcastic tweet;
- an expression of resignation that could be read as neutral or mildly negative;
- the resulting label may not reflect genuine consensus.

You have to keep this in mind when looking at classifier results. If a model gets a tweet "wrong", I can't help but ask if the gold label was ever solid to begin with. A lot of times, it wasn't.

I ran random undersampling (random_state=82171165) to even out the training classes. Ended up with 21,279 tweets—7,093 per class. The validation split? 2,000 tweets, and that's my go-to evaluation set from here on out. The test split (12,284 tweets) just sits on the bench. If you want all the per-class counts, check notebook section 2.5.

The validation set is lopsided: 312 negative, 869 neutral, 819 positive. That stings. negative has less than half the examples of positive, and not even 40% as many as neutral. The moment I spotted this, I knew interpreting negative results would be a pain. Bad performance there? Could be label noise. Could be lousy features. Or maybe just not enough examples to judge. Most likely, it's all of that, tangled together.

And here's another thing: the error bars are wild. With only 312 negative tweets, any F1 score for that class is shaky. Compare that to 869 for neutral. When I see a gap between negative and neutral F1, I get suspicious—is this real, or just statistical static? I can't ignore the difference, but I don't trust it blindly either.

Tweets are short, informal, and noisy: hashtags, @-mentions, URLs, slang, emoji, and abbreviations are common. They are also highly context-dependent. The same words can carry different sentiments depending on:

- the event being discussed;
- who is being addressed;
- whether the author is being ironic.

Tweets about a sports loss sometimes toss in "great" with total sarcasm—I saw a bunch like that when I skimmed the data. Others, aimed at celebrities, throw around "love" and mean it. These quirks make Twitter sentiment a tougher nut to crack than product reviews or news. Context slips and slides. Language is less predictable. I never did run a side-by-side test, but my gut tells me it's trickier here.

I grabbed a random 10 tweets from training, just to see. A lot were, frankly, a toss-up even for me. Like one: a concert announcement, totally neutral words, but it got a positive label. I get it—the author was clearly happy about it—but if you only read the text, there's no obvious clue. Another tweet: mild frustration, all indirect—no swearing, no yelling, just a quick complaint—but it got called neutral. I'd have stuck it in negative. These edge cases are everywhere in Twitter. I expect the classifier to flounder on them, same as a cautious human would.

3 Model

I ran two classifiers—logistic regression and a shallow decision tree—across six feature setups and two task types. That makes four total runs. Both use the same scikit-learn pipeline, balanced training, and I always check performance on the validation split. Figure 1 in the notebook shows the pipeline.

Here's how it goes: First, TextCleaner tidies up whitespace. Then, SpacyPreprocessor tokenises each tweet using the en_core_web_sm spaCy model and parks the results in an SQLite feature store. Next, FeatureUnion builds the feature matrix from whatever feature blocks are active that run. Last, the classifier gets the matrix and does its thing. Swapping out feature configs is simple—preprocessing stays locked down.

Logistic regression is configured with max_iter=5000 and random_state=42. For the tree, I set max_depth=3 and random_state=42. That shallow depth means no more than seven leaf nodes:

- it is forced to identify only the most discriminating features;
- the resulting model is small enough to visualise;

- and small enough to interpret directly.

I made the tree shallow on purpose. Sure, a deeper tree would probably do better on F1. But here, I care way more about understanding what the model latches onto than getting the highest score. Seven nodes? I can print and read those. The max_depth=3 setting is as much a diagnostic tool as a classifier. Yes, it hurts performance. I'll get to that in Section 5.

3.1 Feature configurations

Six feature configurations are tested. They are designed to progress from simple token-count representations through to richer, more linguistically informed features, so that each step in complexity can be assessed against the baseline.

Unigrams. I use a TokensVectorizer to grab counts for the top 200 unigrams, after tossing out stop words and punctuation, with everything lowercased. It's the barest bag-of-words setup. Word order? Gone. Negation? No chance. Context? Forget it. I figured this would flop, and honestly, it sort of does. Still, it's the floor—if a model can't beat this, something's really wrong.

Bigrams. Same vectorizer, but now I'm after the 200 most common bigrams. They give a taste of local context—"not good" isn't "good"—but that's about it. Twitter's language is wild. Most bigrams are rare. Maybe "really love" pops up a lot, but "really enjoy" or "absolutely love"? Might see them once, if at all. Top-200 just gets the common stuff, and honestly, those aren't always the best for sentiment.

Unigrams with POS tags (uni+pos). Here I mash together the top 100 unigram counts (scaled) and POS tag sequence features from a POSVectorizer via FeatureUnion. Why bother? Sometimes grammar sneaks in a sentiment signal that just counting words misses. Think: intensifiers, negations, the way adjectives pile up. Negation is a classic bag-of-words failure—"not" flips "good", but to a unigram model, they're unrelated. POS tags don't fix everything, but bring a little more structure to the party.

Text statistics (textstats). This one's interesting: a TextstatsTransformer grabs things like tweet length, punctuation counts, and similar surface features, scaled with StandardScaler. negative tweets often use more punctuation and exclamation marks. Short tweets are usually pure reaction. Long ones drift toward neutral. I doubted textstats would shine solo but figured it might help in a combination. Running it alone is a diagnostic choice—you see exactly what these surface features catch, and what slides by.

VADER lexicon (lexicon). A LexiconCountVectorizer counts matches against the VADER sentiment lexicon, scaled with StandardScaler. VADER was designed specifically for social media sentiment (Hutto and Gilbert, 2014):

- it encodes human judgements about which words are positive, negative, and neutral;
- it includes common social media expressions;
- it includes emoticons and slang.

This is probably the most directly relevant feature set for this task. I expected it to perform well. The key question is whether the lexicon covers enough of the actual vocabulary in the tweet_eval dataset, which was collected later than the original VADER development set.

Embeddings. With Model2VecEmbedder, I generate dense vector representations of each tweet using a pre-trained embedding model. These should capture semantic similarity—stuff bag-of-words just can't see. Two tweets, totally different words, same vibe? Their vectors should be close. But does this work with short, noisy tweets? Pre-trained embeddings soak up the quirks of their training corpus. If it's general text, not Twitter, maybe they miss the slang. Of all these features, this is the one I found most interesting to pick apart. Feels like the deepest water.

3.2 Task variants

The 3-class task (negative, neutral, positive) is the primary task. The binary task (negative vs positive only, with neutral tweets excluded) was more of a diagnostic run. My instinct was to start with the simpler binary task and then ask what happens when neutral is added back. Here's the idea: if performance jumps substantially in the binary setting, then the trouble is concentrated at the neutral boundary. If not, the mess is spread somewhere else. That's directly relevant to sub-question 4.

For the binary task, I grabbed a whole new, independently undersampled training and test split. The binary training set contains 14,186 tweets (7,093 per class), and the binary test set contains 1,131 tweets.

4 Results

The results are organised into four runs:

- RUN 1: 3-class $\times$ logistic regression;
- RUN 2: 3-class $\times$ decision tree;
- RUN 3: binary $\times$ logistic regression;
- RUN 4: binary $\times$ decision tree.

For each run, classification reports and confusion matrices are shown for all six feature configurations. A summary table and a 3-class vs binary comparison table are provided at the end of this section.

4.1 RUN 1: 3-class classification, logistic regression

Logistic regression, six feature configs. The reports and confusion matrices in RUN 1 show it all. The unigram baseline hit macro F1 of 0.447. That's my bar for the rest.

At first, I didn't think raw word counts would do this well, but then—words like "love", "hate", "great", and "terrible" are right in the top 200 and really shout sentiment. The model's not just rolling dice. For a totally random guesser, macro F1 would be 0.333. We are well above that.

But the unigram model is blind to negation, context, and words that float between classes. Stuff like "day", "time", "people"—all over the place in positive and negative tweets—get near-zero weights. The model just latches onto a handful of words tied to one sentiment. Good for a baseline, but there's a lot left unexplained.

VADER lexicon was built for this job. It bakes in how people judge sentiment words and grabs social-media slang you won't find in old-school vocabulary lists. I expected it to outperform the unigram baseline, and the results are worth looking at closely when assessing which feature type captures the most useful signal. If it only boosts negative precision, maybe it's just good for spotting strong negative language and not much else.

Embeddings are supposed to be the most semantically rich. In theory, they grab meaning beyond just counting words. But do they actually do better on noisy, short tweets? That's a real question, and the answer from this dataset says a lot about the limits of general-purpose semantic representations.

4.2 RUN 2: 3-class classification, decision tree

The decision tree flopped, way worse than logistic regression in almost every setup. With just unigrams and max_depth=3, its macro F1 was 0.277—barely better than random chance at 0.333. It mostly spits out "neutral" for everything. Recall for neutral? Almost perfect. negative and positive? Barely registers. That's what you get with a shallow tree: "predict the most common class" is the least-bad rule when only three splits are available.

But it's not just weaker—it fails in a totally different way. Logistic regression spreads its mistakes around. The tree just gives up and uses a near-trivial heuristic. Three levels deep isn't enough for anything subtle. Maybe the features just don't offer clean single-variable splits, either. Sentiment isn't a simple threshold on one word count.

Accuracy for the tree? 0.474. Logistic regression? 0.470. Practically the same. But this is exactly why accuracy is the wrong metric for imbalanced multi-class problems. You can overwhelmingly predict the majority class and look fine on paper, but you're useless for the others. Macro F1 catches this; accuracy hides it. Running both models side by side on this dataset taught me more than any textbook example.

4.3 RUN 3 and RUN 4: binary task

With the binary task, I tossed out all the neutral tweets and just asked the model to distinguish negative from positive. If all the trouble was really concentrated at the neutral boundary, this should make things a lot easier. I expected a significant gain.

The gain table at the end of Section 4 shows how macro F1 changes from 3-class to binary for each feature configuration. A large positive gain for most configurations would confirm that neutral is the primary source of difficulty. A small or inconsistent

gain would suggest that the negative–positive boundary is also problematic, possibly because of label inconsistency at both boundaries.

The decision tree acts differently in the binary setting. Without the neutral class as a majority-class anchor, the tree must find a split that distinguishes negative from positive. This could mean a more useful model—maybe recall balances out more. Or maybe it just shifts which class gets over-predicted. Depends on the features. This is one of the results I was most curious to see.

5 Discussion

5.1 Overall performance

Logistic regression baseline: macro F1 of 0.447 on the 3-class task. Not cracking 0.5, but honestly, that's about what I expected, given the features and the known difficulty of the task. A score under 0.5 doesn't look great, but it's not meaningless. The model isn't just guessing. That matters. With this much annotation noise and the headaches from the neutral class, a macro F1 in the 0.4–0.5 range feels like a plausible baseline.

A random classifier on a balanced three-class problem would achieve macro F1 of 0.333. A model that always predicts the majority class gets high recall for one class but near-zero for the rest—macro F1 drops below 0.333. So 0.447 from logistic regression is comfortably above both floors. The real question isn't whether the classifier works at all. It's why it doesn't work better, and what that says about the data.

The decision tree's macro F1 was 0.277—tough to defend. Accuracy? 0.474, basically the same as logistic regression (0.470). That just shows how useless accuracy is here. With 43.5% neutral in the test set, a model can just say "neutral" for every input and score 43.5% accuracy without learning anything. The tree isn't quite that bad, but it's close. Macro F1 exposes this; accuracy conceals it.

5.2 Feature comparison

The six feature configurations produce different results, and some patterns jump out. Unigrams set the floor—everything else has to beat them. Bigrams bring in a little local context, but Twitter's messy, so most bigrams are rare. The uni+pos combination adds grammatical structure. Text statistics capture surface properties that may correlate with emotional intensity. VADER lexicon features are purpose-built for this task. Embeddings provide the richest semantic representation, though whether that richness translates into better predictions on short noisy text is not obvious in advance.

I expected the VADER lexicon to be the top performer. It's made for social media, bakes in human judgements about sentiment, and grabs slang, emoticons, and expressions that general vocabulary counts miss. Hutto and Gilbert (2014) demonstrated its effectiveness on Twitter data, and this task is almost identical in character to those used for evaluation.

Embeddings capture semantic similarity, so synonyms and paraphrases of sentiment expressions should map to similar feature vectors. But if those embeddings come from

generic text rather than social media, they may miss Twitter's informal shorthand. A tweet like "can't even rn" expresses frustration that a general-purpose embedding model may not reliably encode as negative.

Text statistics deserve a look. Tweet length, punctuation count, capitalisation patterns—these are blunt tools for guessing emotion, but not useless. Short tweets are usually a snap reaction. High punctuation density often accompanies strong emotion. I didn't expect textstats to shine on their own, but running them solo is a diagnostic choice: it shows what surface quirks can do, and sets a bar for combination models.

Something that jumped out when I compared feature configurations: the per-class ordering barely budges. In logistic regression, per-class F1 almost always lines up as negative < positive < neutral, even when the absolute numbers change. That says to me the difficulty is a property of the data, not the features. negative is just harder to classify, regardless of what model or features you throw at it.

5.3 What the binary task reveals

Sub-question 4 asks whether removing neutral substantially improves performance. The binary task results go straight at the heart of label quality: was neutral the real problem, or is the trouble more general?

If dropping neutral gives a big, consistent boost across classifiers and feature types, that says the neutral label is underspecified—a mushy middle ground that's genuinely hard to separate from mild negative and mild positive expression. That fits what I know about the SemEval annotation process, where neutral was the default for tweets that did not clearly express a directional sentiment.

But if there's only a small gain—or none at all for some configurations—then the mess is at the negative–positive boundary too, or the feature representations just aren't adequate to capture the distinction even without neutral in the way.

The gain comparison table in Section 4 lays out macro F1 for both setups and the difference. I find this view more informative than raw F1 figures, because it speaks directly to the label quality question rather than just classifier performance. Big, consistent gains point to neutral as the culprit. Small or variable gains suggest the difficulty is more distributed.

One more consideration: the binary task uses a different, smaller test set—the original validation set minus all neutral tweets. Any macro F1 gain might partly reflect this compositional change rather than purely an easier task. I keep that in mind when reading the results, though the direction of effect is unlikely to be reversed by it.

5.4 Class-level analysis

The 3-class LR results are not balanced across classes. Positive tweets got the best F1, with high precision but only middling recall—the model doesn't call many tweets positive, but when it does, it's usually right. negative is the worst performer: low precision, moderate recall. The model tags too many tweets as negative, and most of those turn out to be neutral or positive. Neutral sits in the middle.

I didn't expect neutral to beat negative, but it did. The conventional view—supported by the course notes and by Kenyon-Dean et al. (2018)—is that neutral is the hardest class because it lacks distinctive lexical markers. The results don't clearly support this. Neutral F1 exceeds negative F1. Maybe that's because neutral is the largest group in the test set, giving the model more evaluation signal. Or maybe neutral tweets genuinely cluster around non-sentiment-loaded vocabulary that shows up well in the top-200 unigrams. negative might just be more varied and context-dependent language-wise.

The decision tree collapse is a separate phenomenon. It's not just weak on negative and positive—it's nearly zero on both. The tree has apparently learned that predicting neutral minimises loss on the training data even after undersampling. That bothers me. It suggests the feature representations don't yield a clean split between neutral and the other classes at any single threshold.

5.5 Label quality evidence

The central question of this project is not simply how good the classifier is, but what classifier performance tells us about the labels. The two are not the same thing. Not even close.

A low macro F1 could mean a lot of things. Maybe the features are inadequate. Maybe the model isn't suited to the task. Maybe the labels are noisy. All three are probably true to some degree here. I genuinely cannot tell which dominates.

negative stands out. Low precision means the model generates many false positive predictions for negative sentiment. If the model predicts negative based on reasonable lexical cues but the gold label is neutral, this suggests annotators labelled borderline tweets as neutral rather than negative. The SemEval annotation guidelines used neutral as the default for tweets that did not clearly express positive or negative sentiment. The neutral label may have introduced systematic ambiguity in this dataset (Kenyon-Dean et al., 2018):

- the category was designed to capture genuine neutrality alongside mild negative expression;
- annotators were uncertain about tweets that fell between the two;
- sentiment is a property of the author's relationship to content, not just of the words used.

A phrase like "not bad at all" could be positive or ironic depending on context. Unigram count features, which discard word order, negation scope, and context, are poorly equipped to capture these distinctions. But sometimes, the trouble isn't the model's fault—it's just genuinely ambiguous:

- even human annotators would disagree on these cases;
- the low classifier performance may be tracking that annotator disagreement;
- rather than merely reflecting model inadequacy.

The low negative precision may be an alignment signal. The model and the annotators share a rough understanding that certain vocabulary is negative, but their labels diverge:

- the model predicts negative, and in many cases the tweet probably is negative in some sense;
- the annotators, however, labelled it neutral;
- possibly because the expression was not strong enough to trigger the positive or negative label.

If I had access to per-annotator votes rather than only the majority-vote outcome, I could check whether these tweets had high disagreement rates. Not having that is a genuine limitation.

5.6 Misclassification analysis

Notebook section 4.2 produces a systematic breakdown of model predictions by true and predicted class. Looking at the misfires is the best way to see where things go off the rails.

Some misclassifications are model failures. A tweet containing strongly negative language predicted as neutral is likely a case where the relevant tokens fell outside the top-200 vocabulary, or where negation flipped the meaning of otherwise neutral words. These are feature engineering problems.

Other misclassifications are more interesting. When I checked tweets the model called negative but the label was neutral, a recurring pattern came up:

- tweets expressing mild dissatisfaction, slight frustration, or low-key complaint;
- the kind of expression that sits at the boundary between neutral and negative in everyday language.

I'd expect human annotators to disagree on these. The model isn't clearly wrong. The label isn't clearly right. That ambiguity is the point.

The same goes for tweets the model calls positive but the label is neutral. Sometimes there's quiet enthusiasm, but the annotator read it as reporting rather than feeling. These borderline cases are exactly what the central research question is about.

Across all feature configurations, the tweets the model consistently misses are the sarcastic, ironic, or highly context-dependent ones. "Great job guys" directed at a losing team could mean anything. No unigram or bigram feature set can resolve this. A human with knowledge of the team and the result would get it immediately. The classifier has only the words. That's a hard ceiling for surface-form features, and it's a real constraint.

This is not a criticism of the approach. It is the nature of the problem. Did annotators have the context they needed when labelling? If they saw tweets in isolation—no thread, no event background—they probably defaulted to neutral when unsure. That would explain a lot of the patterns observed here.

5.7 Limitations and future directions

The analysis has several limitations.

The feature configurations tested here are all applied independently. No multi-feature combinations are evaluated. A combined model may outperform any single configuration:

- for example, unigrams plus VADER lexicon plus text statistics;
- the interaction effects between feature types are not captured here.

Mixing features is the obvious next step, and the pipeline architecture supports it directly through FeatureUnion. The cost is missing those interactions; the benefit is a clean additive comparison of what each feature type contributes.

Setting max_depth=3 made the tree readable, but it kneecapped performance. A deeper tree might score higher, but then interpretability goes out the window and overfitting becomes a real risk. I went shallow on purpose—for diagnostics, not just accuracy.

The embeddings used here are from a pre-trained general-purpose model. Fine-tuning on Twitter data, or using something like BERTweet (pre-trained on 850 million tweets), would likely produce more useful tweet embeddings. General models just don't always capture the quirks of short, informal text.

The analysis is constrained to the validation split. The held-out test split (12,284 tweets) has not been used and would provide a more reliable estimate of generalisation performance. The validation set has only 312 negative examples, which limits the precision of per-class estimates for that class. Those wide confidence intervals mean negative F1 should be taken as indicative rather than definitive.

Finally, the crowdsourced annotation process introduces noise that no classifier can remove. The gold standard labels represent majority-vote outcomes, not ground truth. Where annotators disagreed, the label is uncertain, and classifier performance on those cases is not a reliable measure of model quality. If I had inter-annotator agreement scores, I could separate the genuinely hard cases from the clearly labelled ones. Without that, I cannot distinguish "model failure" from "tweet nobody would ever agree on." That is probably the single most valuable extension of this analysis.

References

Barbieri, F., Camacho-Collados, J., Espinosa-Anke, L., and Noves, L. (2020). TweetEval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1644–1650. Association for Computational Linguistics.

Hutto, C. J. and Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM 2014)*. AAAI Press.

Kenyon-Dean, K., Ahmed, E., Bhosale, S., Brooks, B., Laframboise, C., Roper, F., Singh, S., Cheung, J. C. K., and Precup, D. (2018). Sentiment analysis: It's complicated! In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1886–1895. Association for Computational Linguistics.

Nash, C. (2015). Atolls in the ocean—canaries in the mine? Australian journalism contesting climate change impacts in the Pacific. *Pacific Journalism Review*, 21(1):79–97.

Rosenthal, S., Farra, N., and Nakov, P. (2017). SemEval-2017 task 4: Sentiment analysis in twitter. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 502–518, Vancouver, Canada. Association for Computational Linguistics.